



PEER-TO-PEER FILE SHARING APPLICATION REPORT

CSCI 21023



CS/2021/026 – DESHAN W.C.

CS/2021/013 – KULANAKA W.A.L.

Source Codes (By Java language)

Sever Code :-

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.nio.file.*;

public class FileServer {

    private static final int PORT = 5001;
    private static final String UPLOAD_DIR = "server_files/";
    private ServerSocket serverSocket;

    public FileServer() {
        // Create upload directory if it doesn't exist
        new File(UPLOAD_DIR).mkdirs();
    }

    public void start() {
        try {
            serverSocket = new ServerSocket(PORT);
            System.out.println("Server started on port " + PORT);

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("New client connected: " + clientSocket.getInetAddress());

                // Handle each client in a new thread
                ClientHandler clientHandler = new ClientHandler(clientSocket);
                new Thread(clientHandler).start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
    }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

```
private class ClientHandler implements Runnable {
```

```
    private Socket clientSocket;  
    private DataInputStream dis;  
    private DataOutputStream dos;
```

```
    public ClientHandler(Socket socket) {  
        this.clientSocket = socket;  
        try {  
            dis = new DataInputStream(socket.getInputStream());  
            dos = new DataOutputStream(socket.getOutputStream());  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
@Override
```

```
public void run() {  
    try {  
        while (true) {  
            String command = dis.readUTF();  
  
            switch (command) {  
                case "UPLOAD":  
                    receiveFile();  
            }  
        }  
    }  
}
```

```

        break;
    case "DOWNLOAD":
        sendFile();
        break;
    case "LIST":
        listFiles();
        break;
    case "EXIT":
        clientSocket.close();
        return;
    }
}
} catch (IOException e) {
    System.out.println("Client disconnected: " + clientSocket.getInetAddress());
}
}

```

```

private void receiveFile() throws IOException {
    String fileName = dis.readUTF();
    long fileSize = dis.readLong();

    FileOutputStream fos = new FileOutputStream(UPLOAD_DIR + fileName);
    byte[] buffer = new byte[4096];
    long remaining = fileSize;
    int read;

    while (remaining > 0 && (read = dis.read(buffer, 0, (int)Math.min(buffer.length,
remaining))) > 0) {
        fos.write(buffer, 0, read);
        remaining -= read;
    }
}

```

```
fos.close();

System.out.println("File received: " + fileName);

dos.writeUTF("File uploaded successfully");

}
```

```
private void sendFile() throws IOException {

    String fileName = dis.readUTF();

    File file = new File(UPLOAD_DIR + fileName);
```

```
    if (!file.exists()) {

        dos.writeBoolean(false);

        return;

    }
```

```
    dos.writeBoolean(true);

    dos.writeLong(file.length());
```

```
    FileInputStream fis = new FileInputStream(file);

    byte[] buffer = new byte[4096];

    int read;
```

```
    while ((read = fis.read(buffer)) > 0) {

        dos.write(buffer, 0, read);

    }
```

```
    fis.close();

    System.out.println("File sent: " + fileName);

}
```

```
private void listFiles() throws IOException {  
    File directory = new File(UPLOAD_DIR);  
    String[] files = directory.list();  
  
    if (files == null) {  
        dos.writeInt(0);  
        return;  
    }  
  
    dos.writeInt(files.length);  
    for (String file : files) {  
        dos.writeUTF(file);  
    }  
}  
  
public static void main(String[] args) {  
    FileServer server = new FileServer();  
    server.start();  
}
```

Client Code :-

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class FileClient {

    private static final String SERVER_IP = "localhost";
    private static final int SERVER_PORT = 5001;
    private static final String DOWNLOAD_DIR = "client_files/";
    private Socket socket;
    private DataInputStream dis;
    private DataOutputStream dos;
    private Scanner scanner;

    public FileClient() {
        // Create download directory if it doesn't exist
        new File(DOWNLOAD_DIR).mkdirs();
        scanner = new Scanner(System.in);
    }

    public void connect() {
        try {
            socket = new Socket(SERVER_IP, SERVER_PORT);
            dis = new DataInputStream(socket.getInputStream());
            dos = new DataOutputStream(socket.getOutputStream());
            System.out.println("Connected to server.");

            showMenu();
        } catch (IOException e) {
            System.out.println("Could not connect to server.");
        }
    }
}
```

```
        e.printStackTrace();
    }
}
```

```
private void showMenu() {
    while (true) {
        try {
            System.out.println("\n=== P2P File Sharing Client ===");
            System.out.println("1. Upload file");
            System.out.println("2. Download file");
            System.out.println("3. List available files");
            System.out.println("4. Exit");
            System.out.print("Choose an option: ");

            int choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
                case 1:
                    uploadFile();
                    break;
                case 2:
                    downloadFile();
                    break;
                case 3:
                    listFiles();
                    break;
                case 4:
                    dos.writeUTF("EXIT");
                    socket.close();
            }
        }
    }
}
```

```

        System.exit(0);
    default:
        System.out.println("Invalid option!");
    }
} catch (IOException e) {
    System.out.println("Connection lost to server.");
    break;
}
}
}

```

```

private void uploadFile() throws IOException {
    System.out.print("Enter file path to upload: ");
    String filePath = scanner.nextLine();
    File file = new File(filePath);

    if (!file.exists()) {
        System.out.println("File does not exist!");
        return;
    }
}

```

```

dos.writeUTF("UPLOAD");
dos.writeUTF(file.getName());
dos.writeLong(file.length());

```

```

FileInputStream fis = new FileInputStream(file);
byte[] buffer = new byte[4096];
int read;

```

```

while ((read = fis.read(buffer)) > 0) {

```

```

        dos.write(buffer, 0, read);
    }

    fis.close();
    String response = dis.readUTF();
    System.out.println(response);
}

private void downloadFile() throws IOException {
    dos.writeUTF("LIST");
    int fileCount = dis.readInt();

    if (fileCount == 0) {
        System.out.println("No files available on server.");
        return;
    }

    System.out.println("\nAvailable files:");
    for (int i = 0; i < fileCount; i++) {
        System.out.println((i + 1) + ". " + dis.readUTF());
    }

    System.out.print("Enter file name to download: ");
    String fileName = scanner.nextLine();

    dos.writeUTF("DOWNLOAD");
    dos.writeUTF(fileName);

    if (!dis.readBoolean()) {
        System.out.println("File not found on server!");
    }
}

```

```

        return;
    }

    long fileSize = dis.readLong();
    FileOutputStream fos = new FileOutputStream(DOWNLOAD_DIR + fileName);
    byte[] buffer = new byte[4096];
    long remaining = fileSize;
    int read;

    while (remaining > 0 && (read = dis.read(buffer, 0, (int)Math.min(buffer.length,
remaining))) > 0) {
        fos.write(buffer, 0, read);
        remaining -= read;
    }

    fos.close();

    System.out.println("File downloaded successfully to " + DOWNLOAD_DIR +
fileName);
}

private void listFiles() throws IOException {
    dos.writeUTF("LIST");
    int fileCount = dis.readInt();

    if (fileCount == 0) {
        System.out.println("No files available on server.");
        return;
    }

    System.out.println("\nAvailable files:");
    for (int i = 0; i < fileCount; i++) {

```

```
        System.out.println((i + 1) + ". " + dis.readUTF());
    }
}

public static void main(String[] args) {
    FileClient client = new FileClient();
    client.connect();
}
}
```

Design & Architecture of the Application

The application consists of two main components: FileServer and FileClient. Both components interact via socket connections, enabling file upload, download, and listing from a remote server.

- **FileServer:** A multi-threaded server that listens for incoming client connections on a specified port (5001). It creates a unique thread for each client to handle file transfers independently. It supports three main commands from clients: UPLOAD, DOWNLOAD, and LIST.
 - **File Storage:** Uploaded files are stored in the server_files directory.
 - **Concurrency:** Each client is handled in a separate thread to ensure that multiple clients can connect and perform operations simultaneously.
- **FileClient:** A command-line client that connects to the server and enables users to upload, download, and list files.
 - **Menu Options:** The client provides a simple menu to choose among file operations.
 - **File Storage:** Downloaded files are saved in the client_files directory.

Challenges faced and solutions

1. Handling multiple clients concurrently without interrupting ongoing transfers.

Solution: The server employs multi-threading by creating a new ClientHandler thread for each connected client. This ensures that one client's file operation does not impact others.

2. Large file transfers may cause issues due to connection timeouts or data loss.

Solution: The code uses a buffered approach (4096-byte chunks) to read and write files, optimizing data transmission and reducing memory usage.

3. Ensuring that the file paths are correctly handled across different systems.

Solution: The code specifies relative paths for server_files and client_files directories and checks for file existence before attempting any transfer.

4. Not been able to connect between two computers at first

Solution: After changing some firewall settings in both computers, both codes worked smoothly.

Instructions to Run the Application

1. **Compile the Code:** Save FileServer.java and FileClient.java in the same directory, then open a terminal in that directory.

```
javac FileServer.java FileClient.java
```

2. **Run the Server:** Start the server in one terminal window.

The server will start listening on port 5001 and create the server_files directory if it doesn't exist.

```
java FileServer
```

3. **Run the Client:** Start the client in a separate terminal.

The client will attempt to connect to the server at localhost on port 5001. Once connected, a menu will display options for uploading, downloading, or listing files on the server.

```
java FileClient
```

4. **Testing File Operations:**

- **Upload a File:**

- From the client menu, choose the upload option by selecting 1.
- Provide the full path of the file you want to upload.
(Eg:- client_files/test4.txt)
- After the upload completes, a message should show to confirm success.

```
=== P2P File Sharing Client ===
1. Upload file
2. Download file
3. List available files
4. Exit
Choose an option: 1
Enter file path to upload: client_files/test4.txt
File uploaded successfully
```

- **Download a File:**

- Choose the download option by selecting 2.
- Enter the exact name of the file as listed on the server.
(Eg:- test1.txt)
- The file will download into the client_files directory and it will show a message after file downloaded successfully.

```
=== P2P File Sharing Client ===
1. Upload file
2. Download file
3. List available files
4. Exit
Choose an option: 2

Available files:
1. car.jpeg
2. test1.txt
3. test2.txt
4. test3.txt
5. test4.txt
Enter file name to download: test1.txt
File downloaded successfully to client_files/test1.txt
```

- **List Available Files:**

- Choose the list option by selecting 3.
- This will display all files currently stored on the server.

```
=== P2P File Sharing Client ===
1. Upload file
2. Download file
3. List available files
4. Exit
Choose an option: 3

Available files:
1. car.jpeg
2. test1.txt
3. test2.txt
4. test3.txt
5. test4.txt
```

5. Exit the Application:

- To exit the client application, select option 4 from the menu.
- The client will disconnect from the server, and both applications will terminate.